

## Tutorial 9

---

### Exercise 1: Modelling

1. Model knapsack as an optimization problem according to the definition on Slide 6 of Lecture 9.
2. Model vertex cover as an optimization problem according to the definition on Slide 6 of Lecture 9.

**Solution:**

1. Recall that the decision variant of Knapsack is defined as

$$\{W, T, w_1, \dots, w_n, c_1, \dots, c_n \in \mathbb{N} \mid \exists b_1, \dots, b_n \in \{0, 1\} \text{ s.t.} \\ \sum_{j=1}^n b_j \cdot w_j \leq W \text{ and } \sum_{j=1}^n b_j \cdot c_j \geq T\}$$

In the optimization variant, we want to maximize (why?) the sum of the values of the items we put in the knapsack, while still satisfying the weight constraint.

Hence, we define:

- The set of instances as

$$I = \{(W, w_1, \dots, w_n, c_1, \dots, c_n) \mid W \in \mathbb{N}, w_j \in \mathbb{N} \text{ for all } j, c_j \in \mathbb{N} \text{ for all } j, \text{ and } n \in \mathbb{N}\}.$$

Note that this definition is equivalent to  $I = \bigcup_{n \in \mathbb{N}} \mathbb{N}^{2n+1}$ , i.e., the set of instances is the set of tuples of natural numbers of odd length.

- Given an instance  $i = (W, w_1, \dots, w_n, c_1, \dots, c_n) \in I$ , a possible solution represents a choice of items to put in the knapsack. In the definition of the decision variant, these are represented by bit vectors  $(b_1, \dots, b_n)$  of length  $n$ .

But, not every such vector represents a *feasible* solution, as it might violate the weight constraint. Hence, the set  $f(i)$  of feasible solutions for  $i$  is

$$\{(b_1, \dots, b_n) \in \mathbb{B}^n \mid \sum_{j=1}^n b_j \cdot w_j \leq W\}.$$

- Let  $i = (W, w_1, \dots, w_n, c_1, \dots, c_n)$  be an instance. We measure the *value* of a feasible solution  $s = (b_1, \dots, b_n) \in f(i)$  by the value of the items chosen by the solution  $s$ , i.e.,

$$m(i, s) = \sum_{j=1}^n b_j \cdot c_j.$$

- Finally, we want to maximize the value of the solution, i.e., we pick  $g = \max$ .

2. Recall that the decision variant of vertex cover is defined as

$$\{(G, k) \mid G \text{ is an undirected graph with } k\text{-vertex cover}\}.$$

In the optimization variant, we are interested in finding the size of a minimal vertex cover.

Hence, we define:

- The set  $I$  of instances is the set of undirected graphs.
- The set of feasible solutions for an instance  $G = (V, E)$  is

$$\{C \subseteq V \mid C \text{ is a } k\text{-vertex cover in } G \text{ for some } k\}.$$

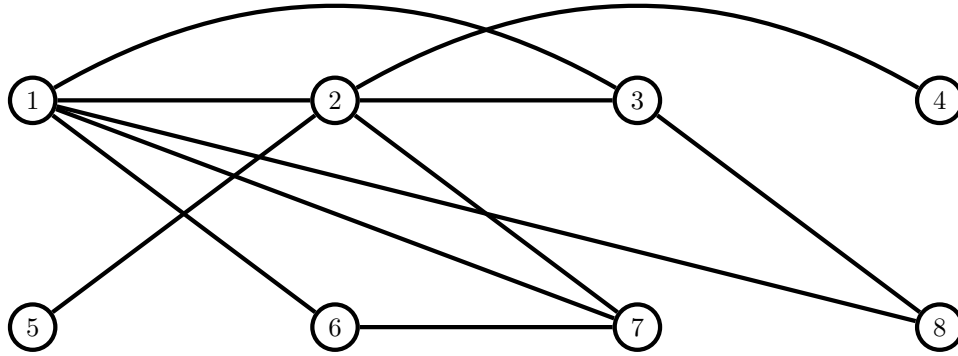
- We measure the value of a vertex-cover by its size, i.e.,

$$m(G, C) = |C|.$$

- We want to minimize, so  $g = \min$ .

**Exercise 2: Vertex cover**

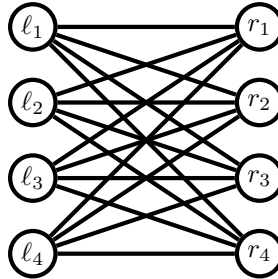
1. Apply the vertex cover approximation algorithm to the following graph.



2. The complete bipartite graph  $K_{m,m}$  is defined as  $K_{m,m} = (V_m, E_m)$  with

- $V_m = \{\ell_j, r_j \mid 1 \leq j \leq m\}$  and
- $E_m = \{(\ell_j, r_{j'}), (r_j, \ell_{j'}) \mid 1 \leq j, j' \leq m\}$ .

Intuitively,  $K_{m,m}$  consists of two disjoint sets of  $m$  vertices, and all edges between these sets, but no edges in the two sets.  $K_{4,4}$  is depicted below.



- (a) What is the size a smallest vertex cover in  $K_{m,m}$ ?
- (b) What is the size of a vertex cover computed by the approximation algorithm presented during the lecture?
- (c) What does this tell us about the approximation ratio of the algorithm?

**Solution:**

1. We use the same heuristic as on the slides to select an edge to cover, i.e., we prioritize vertices with many edges.
  - (a) We cover the edge  $(1, 7)$  by adding both 1 and 7 to  $C$ . This covers the edges  $(1, 2), (1, 3), (1, 6), (1, 7), (1, 8), (6, 7), (2, 7)$ .  
So, after the first iteration, we have  $C = \{1, 7\}$  and  $E' = \{(2, 3), (2, 4), (2, 5), (3, 8)\}$ .
  - (b) We cover the edge  $(2, 3)$  by adding both 2 and 3 to  $C$ . This covers the edges  $(2, 3), (2, 4), (2, 5), (3, 8)$ .  
So, after the second iteration, we have  $C = \{1, 2, 3, 7\}$  and  $E'$  is empty, i.e., the algorithm terminates.

2. (a) The size of a smallest vertex cover is  $m$ , e.g., the one containing all  $m$  vertices  $\ell_j$  on the left side: every edge is incident to such a vertex, so this is indeed a vertex cover.

Now, consider an arbitrary set  $C$  with strictly less than  $m$  vertices. We show that  $C$  is not a vertex cover, showing that  $m$  is indeed the minimum.

As  $|C| < m$ , there is a  $j$  such that  $\ell_j$  and  $r_j$  are not in  $C$ . Hence, the edge  $(\ell_j, r_j) \in E_m$  is not covered by  $C$ , i.e., it is indeed not a vertex cover.

- (b) We show that the algorithm selects all  $2m$  vertices.

We begin by noting that after every iteration of the algorithm, the value of the variable  $C$  satisfies

$$|C \cap \{\ell_j \mid 1 \leq j \leq m\}| = |C \cap \{r_j \mid 1 \leq j \leq m\}|,$$

i.e., at every time it has picked as many vertices on the left side as it has on the right side. This is true, as every edge of the graph has one vertex in  $\{\ell_j \mid 1 \leq j \leq m\}$  and on vertex in  $\{r_j \mid 1 \leq j \leq m\}$  and, once a vertex is added to  $C$ , no edge incident to it is selected later.

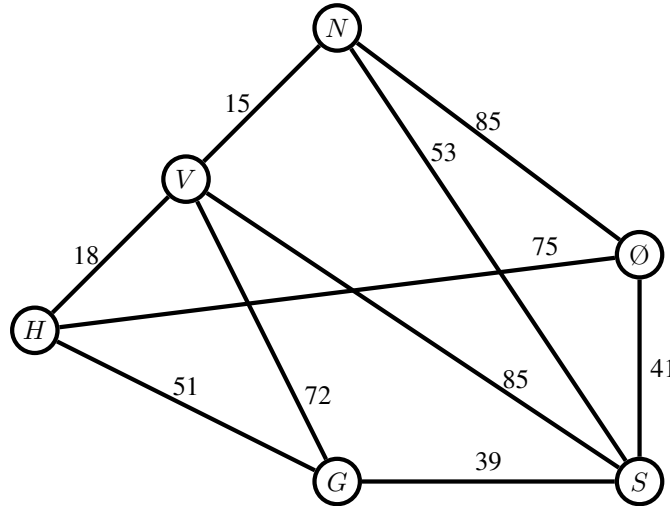
Now, consider the output  $C$  of the algorithm, and, towards a contradiction, assume that  $C \neq V_m$ , i.e., there is a vertex not in  $C$ . Then, by the property above, we conclude that at least two vertices are not in  $C$ , one of the form  $\ell_j$  for some  $1 \leq j \leq m$  and one of the form  $r_{j'}$  for some  $1 \leq j' \leq m$ .

But then the edge  $(\ell_j, r_{j'})$  is not covered by  $C$ , i.e.,  $C$  is not a vertex cover. This is a contradiction, as the algorithm outputs a vertex cover.

- (c) It is at least 2, i.e., the claimed upper bound of 2 presented in the lecture is tight.

### Exercise 3: Minimum spanning trees

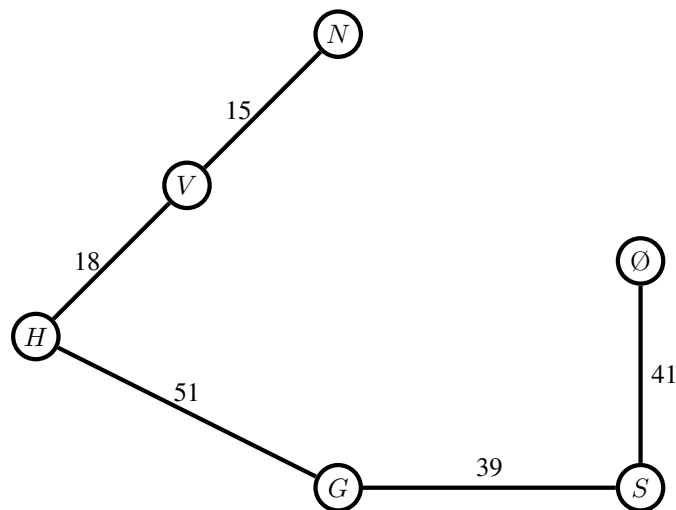
Compute an MST for the following graph using Prim's algorithm.



#### Solution:

- Initially, all vertices are in  $Q$  and have value  $\infty$ , and all successors are undefined.
- All vertices have value  $\infty$ , so we start at vertex  $H$  by removing it from  $Q$ . This updates the values/-successors of the following vertices:
  - $V$ : value 18, successor  $H$ .
  - $G$ : value 51, successor  $H$ .
  - $O$ : value 75, successor  $H$ .

3. The vertex with the lowest value is  $V$ , so we remove it from  $Q$ . This updates the values/successors of the following vertices:  
 $N$ : value 15, successor  $V$ .  
 $S$ : value 85, successor  $V$ .  
The following values/successors are unchanged:  
 $G$ : value 51, successor  $H$ .  
 $\emptyset$ : value 75, successor  $H$ .
4. The vertex with the lowest value is  $N$ , so we remove it from  $Q$ . This updates the values/successors of the following vertices:  
 $S$ : value 53, successor  $N$ .  
The following values/successors are unchanged:  
 $G$ : value 51, successor  $H$ .  
 $\emptyset$ : value 75, successor  $H$ .
5. The vertex with the lowest value is  $G$ , so we remove it from  $Q$ . This updates the values/successors of the following vertices:  
 $S$ : value 39, successor  $G$ .  
The following values/successors are unchanged:  
 $\emptyset$ : value 75, successor  $H$ .
6. The vertex with the lowest value is  $S$ , so we remove it from  $Q$ . This updates the values/successors of the following vertices:  
 $\emptyset$ : value 41, successor  $S$ .
7. The vertex with the lowest value is  $\emptyset$ , so we remove it from  $Q$ . This was the last vertex, so the algorithm terminates and yields the following minimal spanning tree:

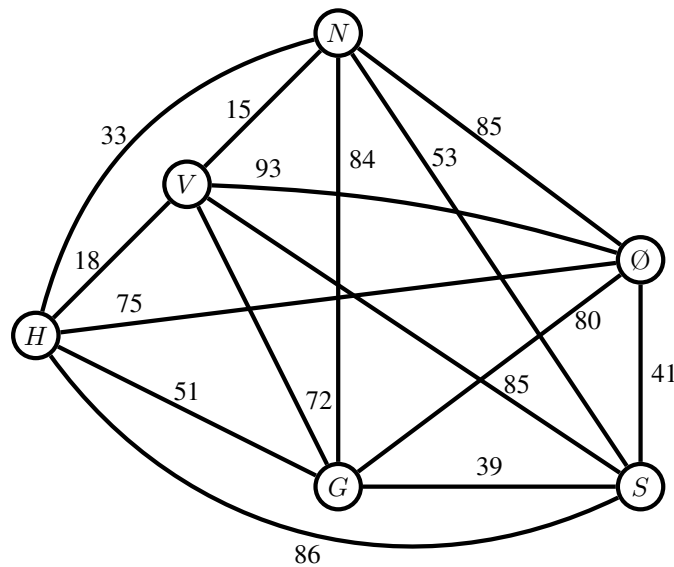


### Exercise 4: Travelling Salesperson

Apply the approximation algorithm for TSP to the graph from Exercise 3.

**Solution:**

We begin by completing the graph (using shortest paths between the missing vertices) yielding the following graph:



The MST computed in Exercise 3 (with weight 164) is still an MST for the completed graph, so we use it.

We start the DFS at vertex  $H$ , yielding the sequence

$$HGS\emptyset SGHVN\emptyset H.$$

Removing duplicates yields the cycle

$$HGS\emptyset VNH$$

with weight 272.

Note that starting at  $N$  (or  $\emptyset$ ) gives a cycle of value 249, i.e., the vertex used to start the DFS can influence the resulting cycle. However, each one of them is at most twice as long as the optimal one.

### Exercise 5: Decision vs. optimization problems

In the lecture, we have claimed that “Decision and optimization are interreducible with polynomial overhead”.

1. Formalize the statement for the decision and the optimization variant of the clique problem.
2. Prove your statement.
3. Can you think of a situation where the interreducibility fails?

**Solution:**

1.  $\text{CLIQUE} \in \text{P}$  if and only if the optimization variant of  $\text{CLIQUE}$  can be solved in polynomial time.

2. We show both directions:

$\Rightarrow$ : Assume  $\text{CLIQUE} \in P$ , i.e., there is a halting DTM  $M$  with time-complexity  $p(n)$  for  $\text{CLIQUE}$ , where  $p(n)$  is some polynomial.

The following Turing machine solves the optimization variant in polynomial time.

On input  $G$ :

1. compute the number  $n$  of vertices of  $G$
2.  $k = 1$
3. Run  $M$  on  $(G, k)$ .
4. If  $M$  accepts and  $k \leq n$ , increment  $k$  and goto step 3.
5. If  $M$  accepts and  $k = n$ , return  $n$ .
6. If  $M$  rejects, return  $k - 1$ .

In the worst, case, we run  $M$   $n$  times, where  $n$  is the number of vertices of  $G$ . Therefore, the algorithm is polynomial in the size of  $G$ . Furthermore, it returns the largest  $k$  such that  $G$  contains a  $k$ -clique: if  $M$  rejects  $(G, k)$  in line 6, then it has accepted for  $k - 1$  and will accept for no  $k' > k$ .

$\Leftarrow$ : Assume the optimization variant of  $\text{CLIQUE}$  can be solved in polynomial time, i.e., there is a Turing machine  $M$  with polynomial time complexity that, given a graph  $G$  returns the maximal  $k$  such that  $G$  has a  $k$ -clique.

Then, the following halting DTM accepts  $\text{CLIQUE}$ .

Given input  $(G, k)$ :

- 1 Run  $M$  on  $G$ .
- 2 If the output of  $M$  is strictly smaller than  $k$  reject, otherwise accept.

This halting DTM runs in polynomial-time, as it calls  $M$  only once, and is obviously correct.

3. The  $\Rightarrow$ -direction depends on the fact that the maximal size of cliques we need to check for is bounded by the size of the graph. In problems where such a bound is too large, this approach does not work.